Formation Vue.js Moderne

Jour 3 — Production & Excellence

Modules 10 à 13 + Projet Final

Composants avancés · API · Performance · Déploiement

📋 Sommaire — Jour 3

#	Module	Contenu	Slides
10	**Composants avancés**	Slots, transitions, portails, dynamique	4–24
11	**Intégration API**	Axios, error handling, caching	25–44
12	**Performance**	Code splitting, lazy loading, bundle	45–64
13	**Production**	Config, sécurité, SEO, monitoring	65–84
—	**Projet final**	Application complète intégrant tout	85–100

> 🕐 Durée estimée : 7h de formation

Module 10

Composants Avancés

Slots · Composants dynamiques · Transitions · Portails

Module 10 — Plan

1. Slots avancés et scoped slots
2. Composants dynamiques
3. Transitions et animations
4. Rendu conditionnel optimisé
5. Teleport (portails)
6. Patterns de performance
7. ✏️ Exercice

10.1 — Slots avancés : rappel et extensions

```
<!-- Slot avec portée (Scoped Slot) -->
<!-- DataTable.vue — passe les données au slot parent -->
<template>
  <table>
    <tbody>
      <tr v-for="row in rows" :key="row.id">
        <slot name="row" :row="row" :index="index">
          <!-- Contenu par défaut -->
          <td v-for="col in columns" :key="col">{{ row[col] }}</td>
        </slot>
      </tr>
    </tbody>
  </table>
</template>

<!-- Parent — personnalise chaque ligne -->
<DataTable :rows="users" :columns="['nom','email']">
  <template #row="{ row, index }">
    <td>{{ index + 1 }}</td>
    <td><strong>{{ row.nom }}</strong></td>
    <td><a :href="`mailto:${row.email}`">{{ row.email }}</a></td>
  </template>
</DataTable>
```

10.2 — Renderless Components

```
// composants/RenderlessToggle.vue
// Composant sans template propre — délègue tout au slot
<script setup lang="ts">
import { ref } from 'vue'

const isOpen = ref(false)
const toggle = () => { isOpen.value = !isOpen.value }
const open   = () => { isOpen.value = true }
const close  = () => { isOpen.value = false }
</script>

<template>
  <slot :isOpen="isOpen" :toggle="toggle" :open="open" :close="close" />
</template>

<!-- Utilisation : logique réutilisée, UI totalement custom -->
<RenderlessToggle v-slot="{ isOpen, toggle }">
  <button @click="toggle">{{ isOpen ? 'Fermer' : 'Ouvrir' }}</button>
  <div v-show="isOpen" class="mon-style-custom">Contenu</div>
</RenderlessToggle>
```

10.3 — Composants dynamiques

```
<script setup lang="ts">
import { ref, defineAsyncComponent } from 'vue'

// Onglets — chaque onglet est un composant
const onglets = [
  { id: 'profil',   label: 'Profil',    component: 'UserProfile' },
  { id: 'securite', label: 'Sécurité',  component: 'UserSecurity' },
  { id: 'notifs',   label: 'Notifs',    component: 'UserNotifications' }
]

const ongletActif = ref('profil')

const composants = {
  UserProfile:       defineAsyncComponent(() => import('./UserProfile.vue')),
  UserSecurity:      defineAsyncComponent(() => import('./UserSecurity.vue')),
  UserNotifications: defineAsyncComponent(() => import('./UserNotifications.vue'))
}
</script>

<template>
  <nav>
    <button v-for="o in onglets" :key="o.id" @click="ongletActif = o.id">
      {{ o.label }}
    </button>
  </nav>
  <!-- KeepAlive préserve l'état des composants inactifs -->
  <KeepAlive>
    <component :is="composants[ongletActif]" />
  </KeepAlive>
</template>
```

10.4 — `<KeepAlive>` : préserver l'état

```
<!-- Sans KeepAlive : état détruit à chaque changement d'onglet -->
<component :is="currentComponent" />

<!-- Avec KeepAlive : état préservé en mémoire -->
<KeepAlive>
  <component :is="currentComponent" />
</KeepAlive>

<!-- Contrôle fin — inclure/exclure des composants -->
<KeepAlive :include="['UserProfile', 'Dashboard']" :max="5">
  <component :is="currentComponent" />
</KeepAlive>

<!-- Hooks spécifiques au KeepAlive -->
<script setup>
import { onActivated, onDeactivated } from 'vue'
onActivated(()   => { console.log("Composant réactivé") })
onDeactivated(() => { console.log("Composant mis en veille") })
</script>
```

10.5 — Transitions : composant `<Transition>`

```
<template>
  <button @click="visible = !visible">Toggle</button>

  <Transition name="slide-fade" mode="out-in">
    <div v-if="visible" key="content">Contenu animé</div>
  </Transition>
</template>

<style>
/* Classes automatiquement ajoutées par Vue */
.slide-fade-enter-active  { transition: all 0.3s ease; }
.slide-fade-leave-active  { transition: all 0.3s ease; }
.slide-fade-enter-from    { opacity: 0; transform: translateX(-20px); }
.slide-fade-leave-to      { opacity: 0; transform: translateX(20px); }

/* Modes : 'out-in' | 'in-out' | default (simultané) */
/* out-in : l'ancien part, puis le nouveau arrive */
</style>
```

10.6 — `<TransitionGroup>` : animer des listes

```
<template>
  <TransitionGroup name="liste" tag="ul">
    <li v-for="item in items" :key="item.id" class="item">
      {{ item.nom }}
      <button @click="supprimer(item.id)">✕</button>
    </li>
  </TransitionGroup>
</template>

<style>
/* Entrée / sortie */
.liste-enter-active, .liste-leave-active { transition: all 0.4s ease; }
.liste-enter-from, .liste-leave-to       { opacity: 0; transform: translateX(30px); }

/* Animation de déplacement des autres items */
.liste-move {
  transition: transform 0.4s ease;
}

/* Important pour que les animations de déplacement fonctionnent */
.liste-leave-active { position: absolute; }
</style>
```

10.7 — Transitions avec JavaScript

```
<Transition
  @before-enter="beforeEnter"
  @enter="enter"
  @after-enter="afterEnter"
  @leave="leave"
  :css="false"
>
  <div v-if="visible">Animé avec JS</div>
</Transition>

<script setup>
function enter(el, done) {
  // Utiliser GSAP, Anime.js, etc.
  gsap.from(el, {
    opacity:  0,
    y:        -30,
    duration: 0.5,
    onComplete: done
  })
}

function leave(el, done) {
  gsap.to(el, { opacity: 0, y: 30, duration: 0.3, onComplete: done })
}
</script>
```

10.8 — `<Teleport>` : portails DOM

```
<!-- Problème : une modale profondément imbriquée -->
<div class="page">
  <section class="contenu" style="overflow: hidden">
    <!-- ❌ La modale est coupée par overflow: hidden -->
    <Modal v-if="showModal" />
  </section>
</div>

<!-- Solution : Teleport — rendu dans <body> mais logique ici -->
<div class="page">
  <section class="contenu" style="overflow: hidden">
    <Teleport to="body">
      <!-- ✅ Rendu dans <body>, hors du overflow -->
      <Modal v-if="showModal" @close="showModal = false" />
    </Teleport>
  </section>
</div>

<!-- Usages typiques de Teleport : -->
<!-- - Modales / Drawers                    -->
<!-- - Tooltips / Popovers                  -->
<!-- - Notifications toast                  -->
<!-- - Menus déroulants contextuels         -->
```

10.9 — Suspense : gestion asynchrone

```
<!-- Suspense gère le chargement de composants asynchrones -->
<Suspense>
  <!-- Composant principal (peut être async) -->
  <template #default>
    <AsyncUserProfile :id="userId" />
  </template>

  <!-- Affiché pendant le chargement -->
  <template #fallback>
    <SkeletonLoader />
  </template>
</Suspense>

<!-- Le composant async peut utiliser await au top level -->
<!-- AsyncUserProfile.vue -->
<script setup lang="ts">
const { data: user } = await useFetch(`/api/users/${props.id}`)
// Pas besoin de gérer loading manuellement ✅
</script>
```

10.10 — v-memo : optimisation des rendus

```
<template>
  <!-- Liste de 1000 items — re-rendu très coûteux -->
  <ul>
    <li
      v-for="item in items"
      :key="item.id"
      v-memo="[item.id, item.selected]"
    >
      <!-- Ce bloc n'est re-rendu QUE si item.id
           ou item.selected change -->
      <ItemLourd :item="item" />
    </li>
  </ul>
</template>
```

```
// shallowRef — réactivité de surface (pas de deep tracking)
import { shallowRef } from 'vue'
```

10.11 — Récapitulatif Module 10

Slots

- Scoped slots `v-slot="{ data }"`
- Renderless components
- Contenu par défaut

Composants dynamiques

- `<component :is="comp">`
- `defineAsyncComponent()`
- `<KeepAlive>` + include/max

Transitions

- `<Transition name="...">`
- `<TransitionGroup>` + move
- Hooks JS + libs (GSAP)

✏️ Exercice — Module 10

Exercice 10.1 — Slots

Exercice 10.2 — Transitions

Exercice 10.3 — Teleport

Exercice 10.4 — Composant dynamique

> 🕐 Durée estimée : 1h00

Module 11

Intégration API et Gestion des Données

Axios · Error handling · Caching · State sync

Module 11 — Plan

1. HTTP clients : Fetch vs Axios
2. Configuration d'Axios
3. Intercepteurs
4. Gestion d'erreurs
5. Caching et optimisation
6. Data validation avec Zod
7. Synchronisation d'état
8. ✏️ Exercice

11.1 — Fetch natif vs Axios

Fetch (natif)

```
const res = await fetch('/api/users', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json',
             'Authorization': `Bearer ${token}` },
  body: JSON.stringify(data)
})
if (!res.ok) throw new Error(res.statusText)
const json = await res.json()
```

Axios

```
const { data } = await axios.post(
  '/api/users',
  data,
  { headers: { Authorization: `Bearer ${token}` } }
)
// json automatique ✅
// erreurs HTTP automatiques ✅
// annulation, timeout, progress ✅
```

11.2 — Configuration d'une instance Axios

```
// src/api/axios.ts
import axios from 'axios'
import { useAuthStore } from '@/stores/auth'
import router from '@/router'

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  timeout: 10_000,
  headers: { 'Content-Type': 'application/json' }
})

// Intercepteur de requête — ajouter le token
api.interceptors.request.use((config) => {
  const auth = useAuthStore()
  if (auth.token) {
    config.headers.Authorization = `Bearer ${auth.token}`
  }
  return config
})

export default api
```

11.3 — Intercepteur de réponse

```
// Intercepteur de réponse — gestion globale des erreurs
api.interceptors.response.use(
  // Succès — retourner la réponse
  (response) => response,

  // Erreur — traitement centralisé
  async (error) => {
    const { status } = error.response ?? {}

    if (status === 401) {
      // Token expiré — tenter le refresh
      try {
        await useAuthStore().refreshToken()
        return api.request(error.config)  // réessayer la requête
      } catch {
        useAuthStore().logout()
        router.push('/login')
      }
    }

    if (status === 403) router.push('/forbidden')
    if (status === 404) router.push('/not-found')

    return Promise.reject(error)
  }
)
```

11.4 — Couche API organisée

```
// src/api/users.ts
import api from './axios'
import type { User, NewUser } from '@/types'

export const usersApi = {
  getAll:  (params?: { page?: number; search?: string }) =>
    api.get<User[]>('/users', { params }),

  getById: (id: number) =>
    api.get<User>(`/users/${id}`),

  create:  (data: NewUser) =>
    api.post<User>('/users', data),

  update:  (id: number, data: Partial<User>) =>
    api.patch<User>(`/users/${id}`, data),

  delete:  (id: number) =>
    api.delete(`/users/${id}`)
}

// Utilisation dans un store Pinia
const { data } = await usersApi.getAll({ page: 1 })
```

11.5 — Gestion d'erreurs structurée

```
// types/api-error.ts
export interface ApiError {
  message: string
  code:    string
  details: Record<string, string[]>  // erreurs de validation
}

// Fonction utilitaire
export function parseApiError(error: unknown): string {
  if (axios.isAxiosError(error)) {
    const data = error.response?.data as ApiError
    return data?.message ?? "Une erreur inattendue est survenue"
  }
  if (error instanceof Error) return error.message
  return "Erreur inconnue"
}

// Dans un store
actions: {
  async fetchUsers() {
    try {
      this.users = (await usersApi.getAll()).data
    } catch (e) {
      this.error = parseApiError(e)
    }
  }
}
```

11.6 — Validation des données avec Zod

```
import { z } from 'zod'

// Définir un schéma de validation
const UserSchema = z.object({
  id:    z.number().positive(),
  nom:   z.string().min(2).max(50),
  email: z.string().email(),
  age:   z.number().min(0).max(150).optional(),
  role:  z.enum(['admin', 'user', 'moderator'])
})

// Inférer le type TypeScript depuis le schéma
type User = z.infer<typeof UserSchema>

// Valider des données reçues de l'API
const validated = UserSchema.safeParse(apiResponse.data)

if (!validated.success) {
  console.error(validated.error.flatten())
} else {
  const user: User = validated.data  // typé et validé ✅
}
```

11.7 — Caching avec TanStack Query (Vue Query)

```
npm install @tanstack/vue-query
```

```
// main.ts
import { VueQueryPlugin } from '@tanstack/vue-query'
app.use(VueQueryPlugin)

// Dans un composant
import { useQuery, useMutation } from '@tanstack/vue-query'

const { data: users, isLoading, error } = useQuery({
  queryKey:  ['users'],
  queryFn:   () => usersApi.getAll().then(r => r.data),
  staleTime: 5 * 60 * 1000  // cache de 5 minutes
})

const { mutate: createUser } = useMutation({
  mutationFn: usersApi.create,
```

11.8 — Annulation de requêtes

```
// Avec AbortController
const controller = new AbortController()

const res = await fetch('/api/search?q=' + query, {
  signal: controller.signal
})

// Annuler si l'utilisateur tape trop vite
onBeforeUnmount(() => controller.abort())

// Avec Axios
const source = axios.CancelToken.source()

api.get('/search', { cancelToken: source.token })

watch(query, () => {
  source.cancel("Nouvelle recherche")
})

// Avec Vue Query — annulation automatique ✅
// Les requêtes sont automatiquement annulées
// si le composant est démonté
```

11.9 — Pagination et données infinies

```
import { useInfiniteQuery } from '@tanstack/vue-query'

const { data, fetchNextPage, hasNextPage } = useInfiniteQuery({
  queryKey: ['posts'],
  queryFn:  ({ pageParam = 1 }) =>
    postsApi.getAll({ page: pageParam }).then(r => r.data),

  getNextPageParam: (lastPage, pages) =>
    lastPage.length === 10 ? pages.length + 1 : undefined
})

// Template
```

```
<template>
  <article v-for="page in data?.pages" v-for="post in page" :key="post.id">
    {{ post.titre }}
  </article>
```

11.10 — Récapitulatif Module 11

Concept	Solution
Instance HTTP	`axios.create()` avec baseURL
Auth token	Intercepteur `request`
Erreurs HTTP	Intercepteur `response`
Couche API	`src/api/*.ts` par ressource
Validation	Zod — schémas + types inférés
Caching	Vue Query — `staleTime`
Annulation	`AbortController` / CancelToken
Pagination	`useInfiniteQuery`

✏️ Exercice — Module 11

Exercice 11.1 — Fetch avec Axios

Exercice 11.2 — Gestion des erreurs

Exercice 11.3 — Intercepteur

> 🕐 Durée estimée : 1h00

Module 12

Performance et Optimisation

Bundle · Lazy loading · Runtime · SSR

Module 12 — Plan

1. Analyser le bundle
2. Code splitting avancé
3. Lazy loading intelligent
4. Optimisation du runtime Vue
5. Stratégies de cache
6. Server-Side Rendering (intro)
7. ✏️ Exercice

12.1 — Analyser le bundle

```
# Visualiser le bundle
npm install -D rollup-plugin-visualizer

# vite.config.ts
import { visualizer } from 'rollup-plugin-visualizer'

export default defineConfig({
  plugins: [
    vue(),
    visualizer({
      filename: 'dist/stats.html',
      open:     true,
      gzipSize: true
    })
  ]
})

# Lancer
npm run build
# → Ouvre stats.html dans le navigateur
```

12.2 — Code splitting et chunks

```
// vite.config.ts — configuration des chunks
export default defineConfig({
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          // Séparer les gros vendors dans leurs propres chunks
          'vendor-vue':   ['vue', 'vue-router', 'pinia'],
          'vendor-axios': ['axios'],
          'vendor-ui':    ['@headlessui/vue', '@heroicons/vue']
        }
      }
    }
  }
})

// Résultat — au lieu d'un gros bundle :
// main.js         (votre code)
// vendor-vue.js   (vue + router + pinia) — mis en cache par le browser
// vendor-axios.js (axios)
// vendor-ui.js    (composants UI)
```

12.3 — Lazy loading intelligent

```
// Routes — toujours en lazy loading
const routes = [
  { path: '/',          component: () => import('@/views/Home.vue') },
  { path: '/dashboard', component: () => import('@/views/Dashboard.vue') }
]

// Composants — lazy loading conditionnel
import { defineAsyncComponent } from 'vue'

// Charger seulement si utilisé
const GrafiquesLourds = defineAsyncComponent(() =>
  import('./GrafiquesLourds.vue')
)

// ❌ Import statique — toujours dans le bundle initial
import GrafiquesLourds from './GrafiquesLourds.vue'

// ✅ Import dynamique — dans un chunk séparé
const modal = defineAsyncComponent(() => import('./Modal.vue'))
```

12.4 — Optimisation des images

```
// vite.config.ts
import { defineConfig } from 'vite'

// Plugin d'optimisation d'images
export default defineConfig({
  build: {
    assetsInlineLimit: 4096,  // images < 4kb → base64 inline
  }
})
```

```
<template>
  <!-- Lazy loading natif des images -->
  <img loading="lazy" :src="imageUrl" :alt="description" />

  <!-- Format WebP avec fallback -->
  <picture>
    <source :srcset="imageWebp" type="image/webp" />
```

12.5 — Performance du runtime Vue

```
<!-- v-once — rendu une seule fois (pas de réactivité) -->
<footer v-once>
  <p>Version {{ version }} — Copyright {{ annee }}</p>
</footer>

<!-- v-memo — mémoïsation conditionnelle -->
<div v-for="item in items" :key="item.id"
     v-memo="[item.id, item.actif, item.selected]">
  <ItemComplexe :item="item" />
</div>

<!-- shallowRef pour les grands datasets -->
<script setup>
import { shallowRef, triggerRef } from 'vue'
const bigList = shallowRef([...thousands_of_items])

function addItem(item) {
  bigList.value.push(item)
  triggerRef(bigList)  // forcer la mise à jour
}
</script>
```

12.6 — Web Workers pour les calculs lourds

```
// worker.ts — s'exécute dans un thread séparé
self.onmessage = (event) => {
  const { data } = event
  const result = computeHeavyTask(data)   // calcul lourd
  self.postMessage(result)
}

// Dans le composant
const worker = new Worker(
  new URL('./worker.ts', import.meta.url),
  { type: 'module' }
)

worker.postMessage(hugeDataset)

worker.onmessage = (event) => {
  result.value = event.data  // réception du résultat
}

onUnmounted(() => worker.terminate())
```

12.7 — Stratégies de cache

```
// Service Worker avec Workbox (via vite-plugin-pwa)
import { VitePWA } from 'vite-plugin-pwa'

export default defineConfig({
  plugins: [
    VitePWA({
      registerType: 'autoUpdate',
      workbox: {
        // Stratégies de cache
        runtimeCaching: [
          {
            urlPattern: /^https:\/\/api\.example\.com\//,
            handler: 'NetworkFirst',   // réseau d'abord, cache si offline
            options: { cacheName: 'api-cache', expiration: { maxAgeSeconds: 3600 } }
          },
          {
            urlPattern: /\.(png|jpg|webp|svg)$/,
            handler: 'CacheFirst',     // cache d'abord
            options: { cacheName: 'images' }
          }
        ]
      }
    })
  ]
})
```

12.8 — SSR avec Nuxt 3 (introduction)

```
# Créer un projet Nuxt 3 (Vue.js + SSR)
npx nuxi@latest init mon-projet-nuxt
```

```
// pages/index.vue — page auto-routée
<script setup lang="ts">
// useFetch de Nuxt — SSR-friendly
const { data: posts } = await useFetch('/api/posts')
</script>
<template>
  <div v-for="post in posts">{{ post.titre }}</div>
</template>
```

Modes de rendu Nuxt :

Mode	Description	Usage

12.9 — Checklist performance

Build :

- [ ] Analyser le bundle (`rollup-plugin-visualizer`)
- [ ] Toutes les routes en lazy loading
- [ ] Composants lourds en `defineAsyncComponent`
- [ ] Images optimisées + WebP + lazy loading

Runtime :

- [ ] `v-memo` sur les listes longues
- [ ] `shallowRef` pour les grands datasets
- [ ] Calculs lourds dans Web Workers
- [ ] `v-once` pour le contenu statique

Réseau :

12.10 — Récapitulatif Module 12

Technique	Impact	Outil
Lazy loading routes	Bundle initial réduit	`() => import()`
`manualChunks`	Meilleur cache navigateur	`vite.config.ts`
`v-memo`	Rendus évités	Template directive
`shallowRef`	Tracking réduit	Composition API
Images WebP	-30% à -80% poids	`<picture>`
Service Worker	Offline + cache	Workbox / PWA
Web Workers	UI non bloquée	Browser API
SSR	SEO + perf perçue	Nuxt 3

✏️ Exercice — Module 12

Exercice 12.1 — Analyse du bundle

Exercice 12.2 — Lazy loading des routes

Exercice 12.3 — v-memo

> 🕐 Durée estimée : 1h00

Module 13

Production et Déploiement

Config · Sécurité · SEO · Monitoring · Documentation

Module 13 — Plan

1. Configuration de production
2. Variables d'environnement
3. Sécurité applicative
4. SEO et métadonnées
5. Monitoring et logging
6. Stratégies de déploiement
7. Documentation
8. ✏️ Exercice

13.1 — Build de production

```
# Build complet
npm run build

# Ce que fait 'vue-tsc && vite build' :
# 1. vue-tsc — vérification TypeScript (sans erreur = go)
# 2. vite build — bundle optimisé dans dist/
```

```
// vite.config.ts — config production
export default defineConfig({
  build: {
    outDir:          'dist',
    sourcemap:       false,     // pas de sourcemaps en prod
    minify:          'esbuild', // minification ultra rapide
    chunkSizeWarningLimit: 500, // alerte si chunk > 500kb
    rollupOptions: {
      output: {
        // Hash dans les noms — cache busting automatique
```

13.2 — Variables d'environnement par environnement

```
# .env.development
VITE_API_URL=http://localhost:3001
VITE_DEBUG=true
VITE_LOG_LEVEL=debug

# .env.staging
VITE_API_URL=https://staging.api.example.com
VITE_DEBUG=false
VITE_LOG_LEVEL=warn

# .env.production
VITE_API_URL=https://api.example.com
VITE_DEBUG=false
VITE_LOG_LEVEL=error
VITE_SENTRY_DSN=https://xxx@sentry.io/xxx
```

```
# Lancer avec un mode spécifique
```

13.3 — Sécurité : en-têtes HTTP

```
# nginx.conf — en-têtes de sécurité essentiels
server {
  listen 443 ssl;

  # Content Security Policy
  add_header Content-Security-Policy
    "default-src 'self';
     script-src 'self' 'nonce-${request_id}';
     style-src 'self' 'unsafe-inline';
     img-src 'self' data: https:;
     connect-src 'self' https://api.example.com";

  add_header X-Frame-Options DENY;
  add_header X-Content-Type-Options nosniff;
  add_header Referrer-Policy strict-origin-when-cross-origin;
  add_header Permissions-Policy "camera=(), microphone=(), geolocation=()";

  # HSTS — forcer HTTPS
  add_header Strict-Transport-Security
    "max-age=31536000; includeSubDomains; preload";
}
```

13.4 — Sécurité : bonnes pratiques Vue.js

```
<!-- ❌ XSS — ne jamais utiliser v-html avec des données utilisateur -->
<div v-html="commentaireUtilisateur"></div>

<!-- ✅ Utiliser l'interpolation normale (échappée automatiquement) -->
<div>{{ commentaireUtilisateur }}</div>

<!-- Si v-html est nécessaire, assainir d'abord avec DOMPurify -->
<script setup>
import DOMPurify from 'dompurify'
const htmlSain = DOMPurify.sanitize(contenuHtml)
</script>
<div v-html="htmlSain"></div>
```

```
// Tokens dans les cookies (httpOnly) pas dans localStorage
// ❌ localStorage — accessible par JS (vulnérable XSS)
localStorage.setItem('token', accessToken)
```

13.5 — SEO avec `@vueuse/head`

```
npm install @unhead/vue
```

```
// main.ts
import { createHead } from '@unhead/vue'
const head = createHead()
app.use(head)
```

```
<!-- Dans chaque page / view -->
<script setup lang="ts">
import { useHead, useSeoMeta } from '@unhead/vue'

useSeoMeta({
  title:          'Mon produit | Mon App',
  description:    'Description de la page pour Google',
  ogTitle:        'Mon produit | Mon App',
```

13.6 — Monitoring avec Sentry

```
npm install @sentry/vue
```

```
// main.ts — initialiser Sentry
import * as Sentry from '@sentry/vue'

Sentry.init({
  app,
  dsn:                import.meta.env.VITE_SENTRY_DSN,
  integrations:       [new Sentry.BrowserTracing({ router })],
  tracesSampleRate:   0.2,    // 20% des transactions tracées
  replaysOnErrorSampleRate: 1.0,

  // Ne pas tracker en développement
  enabled: import.meta.env.PROD
})

// Capturer une erreur manuellement
Sentry.captureException(new Error("Paiement échoué"))
```

13.7 — Déploiement : Netlify

```toml
# netlify.toml
[build]
  command   = "npm run build"
  publish   = "dist"

[build.environment]
  NODE_VERSION = "20"

# Redirection pour le SPA routing
[[redirects]]
  from   = "/*"
  to     = "/index.html"
  status = 200

# En-têtes de sécurité
[[headers]]
  for = "/*"
  [headers.values]
    X-Frame-Options       = "DENY"
    X-Content-Type-Options = "nosniff"
    Cache-Control         = "public, max-age=31536000, immutable"
```

13.8 — Déploiement : Docker

```dockerfile
# Dockerfile — build multi-stage
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine AS production
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

```yaml
# docker-compose.yml
services:
  frontend:
```

13.9 — Documentation du projet

README.md minimal :

```
# Nom du projet

## Description
Ce que fait l'application en 2-3 phrases.

## Installation
\`\`\`bash
npm install && npm run dev
\`\`\`

## Variables d'environnement
| Variable | Description | Requis |
|----------|-------------|--------|
| VITE_API_URL | URL de l'API backend | ✅ |
| VITE_SENTRY_DSN | DSN Sentry | ❌ |

## Architecture
Décrire les choix techniques importants.
```

13.10 — Checklist avant mise en production

Code :

- [ ] `npm run build` sans erreur TypeScript
- [ ] `npm run lint` → 0 erreur
- [ ] Tests → 80%+ de couverture
- [ ] Pas de `console.log` oubliés

Config :

- [ ] `.env.production` avec valeurs réelles
- [ ] Sourcemaps désactivés en prod
- [ ] En-têtes de sécurité configurés

Monitoring :

13.11 — Récapitulatif Module 13

Build

- `npm run build`
- Hash = cache busting
- Sourcemaps OFF en prod

Sécurité

- CSP + en-têtes HTTP
- Pas de v-html non sanitisé
- Tokens en httpOnly cookies

SEO

- `useSeoMeta()` par page
- Open Graph
- Sitemap dynamique

✏️ Exercice — Module 13

Exercice 13.1 — Variables d'environnement

Exercice 13.2 — Déploiement

Exercice 13.3 — SEO & Métadonnées

Exercice 13.4 — Documentation du projet

> 🕐 Durée estimée : 1h15

🎯 Projet Final

Application Vue.js Complète

Intégration de tous les modules

Présentation du projet final

Objectif : Développer une application de gestion de tâches avancée qui intègre l'ensemble des compétences de la formation.

Fonctionnalités attendues :

- Authentification (login / logout / persistance)
- CRUD des tâches avec catégories et priorités
- Filtres, recherche et pagination
- Interface animée et accessible
- Déployée en ligne et documentée

Durée : 3 heures (fin de journée)

Architecture du projet final

```
src/
├── api/
│   ├── axios.ts          # instance configurée
│   └── tasks.ts          # couche API tâches
├── components/
│   ├── ui/               # AppButton, AppInput, AppModal
│   └── tasks/            # TaskCard, TaskList, TaskForm
├── composables/
│   ├── useTasks.ts       # logique tâches
│   └── useDebounce.ts    # recherche différée
├── stores/
│   ├── auth.ts           # authentification
│   └── tasks.ts          # état global tâches
├── views/
│   ├── LoginView.vue
│   ├── DashboardView.vue
│   └── TaskDetailView.vue
└── router/index.ts       # routes avec guards
```

Checklist projet final

Jour 1 — Socle

- [ ] Projet créé avec Vite + TypeScript + Pinia + Router
- [ ] Composants UI réutilisables (`AppButton` , `AppInput`)
- [ ] Layout avec navigation

Jour 2 — Fonctionnalités

- [ ] Store `useAuthStore` + guard de route
- [ ] Store `useTasksStore` + actions CRUD
- [ ] Formulaire de tâche avec validation Zod

Jour 3 — Qualité & Production

- [ ] Tests unitaires (composables) ≥ 80 % coverage

Critères d'évaluation

any	
Architecture composants	20
Vue Router — guards + lazy loading	10
Performance — bundle analysé	10
Documentation (README)	5

🎉 Félicitations !

Formation Vue.js Moderne — Complète

Vous maîtrisez maintenant :

JavaScript ES6+ · TypeScript · Vite

Composants SFC · Composition API

Vue Router · Pinia

Tests Vitest + Cypress

Performance · Production · Déploiement

Ressources pour continuer :

`vuejs.org` · `pinia.vuejs.org` · `router.vuejs.org`

`vueuse.org` · `nuxt.com` · `vitest.dev`

📋 Index des slides — Jour 3

Slide	Contenu
1	Couverture
2	Sommaire
3	Module 10 — chapitre
4–15	Composants avancés, transitions, Teleport
16	Récap Module 10
17	Exercice Module 10
18	Module 11 — chapitre
19–30	Axios, intercepteurs, Zod, Vue Query
31	Récap Module 11

10.12 — Accessibilité (a11y) dans Vue.js

```
<template>
  <!-- Bouton avec label accessible -->
  <button
    :aria-expanded="isOpen"
    :aria-controls="menuId"
    @click="toggle"
  >
    Menu <span aria-hidden="true">▾</span>
  </button>

  <nav :id="menuId" :aria-hidden="!isOpen" role="navigation">
    <RouterLink v-for="item in items" :key="item.to" :to="item.to">
      {{ item.label }}
    </RouterLink>
  </nav>

  <!-- Focus trap dans les modales -->
  <div role="dialog" aria-modal="true" aria-labelledby="modal-title">
    <h2 id="modal-title">Confirmation</h2>
    <button @click="confirmer">Confirmer</button>
    <button @click="fermer">Annuler</button>
  </div>
</template>
```

10.13 — Rendu côté serveur partiel (SSR islands)

```
// Pattern "Islands Architecture" avec Nuxt
// Certains composants s'hydratent à la demande

// Hydratation paresseuse — seulement quand visible
<LazyComposantLourd />
// Nuxt auto-préfixe Lazy pour les composants

// Hydratation au scroll
const AsyncHeavy = defineAsyncComponent({
  loader: () => import('./Heavy.vue'),
  hydrate: hydrateOnVisible({ rootMargin: '200px' })
})

// Hydratation au survol
const AsyncMenu = defineAsyncComponent({
  loader: () => import('./Menu.vue'),
  hydrate: hydrateOnInteraction('mouseenter')
})
```

11.11 — WebSockets avec Vue.js

```
// composables/useWebSocket.ts
import { ref, onUnmounted } from 'vue'

export function useWebSocket(url: string) {
  const ws       = new WebSocket(url)
  const messages = ref<string[]>([])
  const status   = ref<'connecting' | 'open' | 'closed'>('connecting')

  ws.onopen    = () => { status.value = 'open' }
  ws.onclose   = () => { status.value = 'closed' }
  ws.onmessage = (event) => {
    messages.value.push(JSON.parse(event.data))
  }

  function send(data: unknown) {
    if (status.value === 'open') {
      ws.send(JSON.stringify(data))
    }
  }

  onUnmounted(() => ws.close())
  return { messages, status, send }
}
```

11.12 — Server-Sent Events (SSE)

```
// composables/useSSE.ts
import { ref, onUnmounted } from 'vue'

export function useSSE(url: string) {
  const data   = ref<unknown>(null)
  const error  = ref<string | null>(null)
  const source = new EventSource(url)

  source.onmessage = (event) => {
    data.value = JSON.parse(event.data)
  }
  source.onerror = () => {
    error.value = 'Connexion perdue'
    source.close()
  }

  onUnmounted(() => source.close())
  return { data, error }
}

// Utilisation pour un dashboard temps réel
const { data: stats } = useSSE('/api/events/dashboard')
// Mise à jour automatique à chaque push serveur ✅
```

12.11 — Optimisation des fonts

```
<!-- index.html — précharger les polices critiques -->
<head>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

  <!-- Préchargement critique — évite le FOUT -->
  <link rel="preload" as="font" type="font/woff2"
        href="/fonts/inter-regular.woff2" crossorigin>

  <!-- Font display swap — texte visible pendant le chargement -->
  <style>
    @font-face {
      font-family: 'Inter';
      src: url('/fonts/inter-regular.woff2') format('woff2');
      font-display: swap;
    }
  </style>
</head>
```

12.12 — Métriques de performance (Core Web Vitals)

Métrique	Signification	Objectif
LCP	Largest Contentful Paint — chargement	< 2.5s
FID/INP	First Input Delay / Interaction to Next Paint	< 200ms
CLS	Cumulative Layout Shift — stabilité visuelle	< 0.1
TTFB	Time to First Byte	< 800ms

```
# Mesurer avec Lighthouse
npx lighthouse https://mon-app.com --output=html

# Mesurer localement
npm install -D @lhci/cli
npx lhci autorun
```

12.13 — Optimisation des requêtes réseau

```
// Prefetch des ressources suivantes
router.beforeEach((to) => {
  // Précharger les composants des routes probables
  if (to.name === 'home') {
    import('@/views/DashboardView.vue')     // précharger en silence
    import('@/views/ProfileView.vue')
  }
})

// Dans les templates — prefetch de liens importants
<RouterLink to="/dashboard" prefetch>
  Dashboard
</RouterLink>

// Resource hints dans index.html
<link rel="dns-prefetch"  href="https://api.example.com">
<link rel="prefetch"      href="/dashboard" as="document">
<link rel="modulepreload" href="/assets/Dashboard-hash.js">
```

13.12 — Gestion des erreurs globales

```
// main.ts — gestionnaire d'erreurs global Vue
app.config.errorHandler = (error, instance, info) => {
  console.error('Erreur Vue:', error)
  console.error('Composant:', instance?.$options.name)
  console.error('Info:', info)

  // Envoyer à Sentry
  Sentry.captureException(error, {
    extra: { component: instance?.$options.name, info }
  })
}

// Gestionnaire d'avertissements (dev seulement)
app.config.warnHandler = (msg, instance, trace) => {
  console.warn('Warning Vue:', msg)
}

// Erreurs non capturées (JavaScript global)
window.addEventListener('unhandledrejection', (event) => {
  Sentry.captureException(event.reason)
})
```

13.13 — Progressive Web App (PWA)

```
// vite.config.ts — vite-plugin-pwa
import { VitePWA } from 'vite-plugin-pwa'

VitePWA({
  registerType: 'autoUpdate',
  manifest: {
    name:        'Mon Application',
    short_name:  'MonApp',
    description: 'Description de l\'application',
    theme_color: '#42b883',
    icons: [
      { src: '/icon-192.png', sizes: '192x192', type: 'image/png' },
      { src: '/icon-512.png', sizes: '512x512', type: 'image/png' }
    ]
  }
})

// Prompt d'installation dans le composant
const { needRefresh, updateServiceWorker } = useRegisterSW()

// Bouton "Mettre à jour" quand nouvelle version disponible
<button v-if="needRefresh" @click="updateServiceWorker()">
  Mettre à jour l'application
</button>
```

13.14 — Internationalisation (i18n)

```
npm install vue-i18n@9
```

```
// i18n/index.ts
import { createI18n } from 'vue-i18n'
import fr from './locales/fr.json'
import en from './locales/en.json'

const i18n = createI18n({
  legacy:  false,  // Composition API
  locale:  navigator.language.startsWith('fr') ? 'fr' : 'en',
  fallbackLocale: 'en',
  messages: { fr, en }
})

// Dans un composant
const { t, locale } = useI18n()
// t('welcome', { nom: 'Alice' }) → "Bienvenue Alice"
```

13.15 — Stratégies de mise à jour et versioning

```
// Détecter une nouvelle version de l'app
const CURRENT_VERSION = import.meta.env.VITE_APP_VERSION

async function checkForUpdates() {
  const res  = await fetch('/api/version')
  const { version } = await res.json()

  if (version !== CURRENT_VERSION) {
    // Informer l'utilisateur
    toast.info('Nouvelle version disponible — rechargez la page')
  }
}

// Vérifier toutes les 5 minutes
setInterval(checkForUpdates, 5 * 60 * 1000)

// Vérifier quand la page reprend le focus
document.addEventListener('visibilitychange', () => {
  if (document.visibilityState === 'visible') checkForUpdates()
})
```

🏁 Récapitulatif de la formation

Les 3 piliers d'une app Vue.js production-ready

```
1. ARCHITECTURE
   ┌──────────────────────────────┐
   │ Composants SFC bien découpés │
   │ Composables pour la logique  │
   │ Pinia pour l'état global     │
   │ Vue Router avec guards       │
   └──────────────────────────────┘

2. QUALITÉ
   ┌──────────────────────────────┐
   │ TypeScript strict            │
   │ Tests ≥ 80% coverage         │
   │ ESLint + Prettier            │
   │ CI/CD automatisé             │
   └──────────────────────────────┘

3. PERFORMANCE
   ┌──────────────────────────────┐
   │ Lazy loading routes et composants│
   │ Bundle analysé et optimisé   │
   │ Cache navigateur (hashes)    │
```

🔗 Ressources et prochaines étapes

Documentation officielle :

- `vuejs.org` — Vue.js 3 (Composition API)
- `router.vuejs.org` — Vue Router 4
- `pinia.vuejs.org` — Pinia
- `vite.dev` — Vite

Aller plus loin :

- `nuxt.com` — SSR / SSG avec Vue
- `vueuse.org` — 200+ composables prêts à l'emploi
- `ui.nuxt.com` — Nuxt UI (composants)
- `tanstack.com/query` — Vue Query

11.13 — Pagination avec curseur

```
// API avec curseur (cursor-based pagination)
// Plus performante que la pagination par offset

interface CursorPage<T> {
  data:       T[]
  nextCursor: string | null
  prevCursor: string | null
  hasMore:    boolean
}

export const useInfiniteMessages = (roomId: string) => {
  return useInfiniteQuery({
    queryKey:  ['messages', roomId],
    queryFn:   ({ pageParam }) =>
      messagesApi.getPage(roomId, { cursor: pageParam }),

    initialPageParam: undefined as string | undefined,
    getNextPageParam: (lastPage) => lastPage.nextCursor ?? undefined,
    getPreviousPageParam: (firstPage) => firstPage.prevCursor ?? undefined,
  })
}
```

12.14 — Compression et tree shaking

```
// Tree shaking — n'importer que ce dont on a besoin

// ❌ Import de tout lodash (400ko)
import _ from 'lodash'
const sorted = _.sortBy(users, 'nom')

// ✅ Import ciblé (seulement sortBy — quelques ko)
import sortBy from 'lodash-es/sortBy'
const sorted = sortBy(users, 'nom')

// ✅ Encore mieux — équivalent natif
const sorted = [...users].sort((a, b) => a.nom.localeCompare(b.nom))
```

```
# Analyser les gains avec esbuild
npm run build 2>&1 | grep -E "kB|MB"
# Avant : dist/assets/index-hash.js   450.23 kB | gzip: 128.45 kB
# Après : dist/assets/index-hash.js   180.12 kB | gzip:  52.30 kB
```

13.16 — Gestion des feature flags

```ts
// composables/useFeatureFlags.ts
import { computed } from 'vue'

// Flags chargés depuis l'API ou la config
const flags = ref<Record<string, boolean>>({})

export async function initFeatureFlags() {
  const res = await fetch('/api/feature-flags')
  flags.value = await res.json()
}

export function useFeatureFlag(flagName: string) {
  return computed(() => flags.value[flagName] ?? false)
}

// Dans un composant
const isNewDashboard = useFeatureFlag('new-dashboard')
const isBetaMode     = useFeatureFlag('beta-mode')
```

13.17 — Logs et observabilité

```ts
// utils/logger.ts
type LogLevel = 'debug' | 'info' | 'warn' | 'error'

const LOG_LEVELS: Record<LogLevel, number> = {
  debug: 0, info: 1, warn: 2, error: 3
}

const MIN_LEVEL = import.meta.env.VITE_LOG_LEVEL as LogLevel ?? 'info'

export const logger = {
  debug: (msg: string, data?: unknown) => log('debug', msg, data),
  info:  (msg: string, data?: unknown) => log('info',  msg, data),
  warn:  (msg: string, data?: unknown) => log('warn',  msg, data),
  error: (msg: string, data?: unknown) => log('error', msg, data),
}

function log(level: LogLevel, msg: string, data?: unknown) {
  if (LOG_LEVELS[level] < LOG_LEVELS[MIN_LEVEL]) return
  console[level](`[${level.toUpperCase()}] ${msg}`, data ?? '')
  if (level === 'error') Sentry.captureMessage(msg)
}
```